TRƯỜNG ĐẠI HỌC THỦY LỢI
KHOA CÔNG NGHỆ THÔNG TIN • BỘ MÔN TRÍ TUỆ NHÂN TẠO

LAB 3

NHẬN DẠNG TIẾNG NÓI NÂNG CAO VỚI HMM

Từ MFCC động đến mô hình Markov ẩn, likelihood, Viterbi, Baum-Welch và giải mã có mô hình ngôn ngữ

Pipeline khái niệm của hệ nhận dạng tiếng nói HMM



1. Mục tiêu và chuẩn đầu ra của Lab

Lab 3 chuyển từ đối sánh mẫu DTW ở Lab 2 sang mô hình xác suất theo thời gian. Sinh viên xây dựng một hệ nhận dạng từ/lệnh tách rời bằng Gaussian HMM, quan sát đường trạng thái Viterbi, thực hiện huấn luyện Baum-Welch thông qua thư viện và đánh giá hệ thống trên dữ liệu chưa thấy.

- Giải thích được cấu trúc HMM $\lambda=(A, B, \pi)$, giả thiết Markov và vai trò của xác suất phát xạ.
- Tạo vector đặc trưng 39 chiều từ MFCC, Δ-MFCC và Δ²-MFCC; hiểu vì sao đặc trưng động bổ sung thông tin cho HMM.
- Tính và diễn giải likelihood $P(X|\lambda)$ bằng thuật toán Forward; tìm chuỗi trạng thái tốt nhất bằng Viterbi.
- Giải thích nguyên lý Baum-Welch/Forward–Backward và ý nghĩa của các đại lượng $\gamma t(i)$, $\xi t(i,j)$.
- Huấn luyện một whole-word HMM cho mỗi từ/lệnh; nhận dạng theo maximum likelihood/MAP và đánh giá bằng confusion matrix.
- Xây dựng một lexicon nhỏ ở mức định dạng dữ liệu; giải thích khi nào lexicon trở nên cần thiết với subword HMM.
- Xây dựng unigram/bigram nhỏ và thử nghiệm re-ranking bằng acoustic score + language-model score.
- So sánh DTW và HMM về khả năng mô hình hóa biến thiên thời gian, nhu cầu dữ liệu, tính mở rộng và độ phức tạp.

1.1. Phân bổ 3 tiết

Thời lượng	Hoạt động	Kết quả
45 phút	Ôn MFCC động; HMM, Forward, Viterbi, Baum-Welch	Hiểu mô hình và công thức cốt lõi
75 phút	Cài đặt pipeline, huấn luyện 5 whole-word HMM, giải mã test set	Hệ nhận dạng chạy được
60 phút	Phân tích Viterbi, so sánh DTW, LM re-ranking, đánh giá và báo cáo	Confusion matrix + nhận xét định lượng

1.2. Kiến thức và kỹ năng tiên quyết

- Lab 2: endpoint detection, MFCC, Euclidean distance, DTW và confusion matrix.
- Xác suất cơ bản: xác suất có điều kiện, Bayes, Gaussian nhiều chiều, log-likelihood.
- Python/NumPy; đọc dữ liệu WAV; thao tác ma trận và danh sách file.
- Không yêu cầu sinh viên tự viết Baum-Welch đầy đủ từ đầu trong 3 tiết; cần hiểu công thức và kiểm tra kết quả huấn luyện của thư viện.

Phạm vi Lab Case study dùng whole-word HMM để giữ bài toán đủ nhỏ cho một buổi 3 tiết. Lexicon và language model được đưa vào như bước mở rộng để kết nối trực tiếp với kiến trúc ASR đầy đủ của Chương 3.

2. Chuẩn bị môi trường và dữ liệu

2.1. Môi trường Python

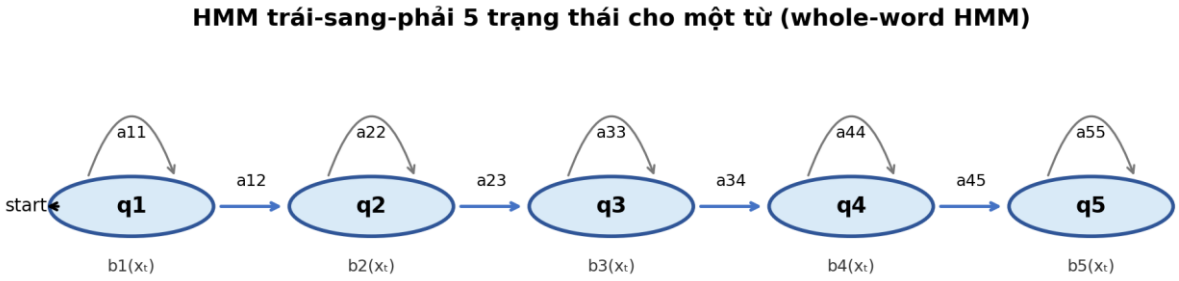
pip install numpy scipy matplotlib librosa soundfile scikit-learn hmmlearn

2.2. Bộ dữ liệu đề xuất

- 5 lệnh tiếng Việt tách rời: "mở", "đóng", "lên", "xuống", "dừng" (có thể thay bằng bộ từ do giảng viên quy định).
- Mỗi lớp tối thiểu 8 utterance/người: 6 train + 2 test. Nếu làm nhóm 2 người, nên có dữ liệu của cả hai để thử nghiệm speaker mismatch.
- WAV mono, Fs = 16 kHz, 16-bit PCM; có 0.2–0.5 s silence ở đầu/cuối để kiểm tra endpoint detection.
- Không để cùng một file xuất hiện ở cả training và test. Với thí nghiệm speaker-independent nhỏ, phải tách theo người nói, không chỉ tách theo file.

```
dataset/
  train/
    mo/mo_01.wav ... mo_06.wav
    dong/dong_01.wav ... dong_06.wav
    len/...
    xuong/...
    dung/...
  test/
    mo/mo_07.wav mo_08.wav
    ...
```

3. Tóm tắt kiến thức và công thức cần dùng



Hình 1. HMM trái-sang-phải: cấu trúc phù hợp với tiến trình theo thời gian của một từ nói.

3.1. Bài toán ASR theo Bayes

Cho chuỗi vector đặc trưng quan sát được $\mathbf{X = (x_1, x_2, \ldots, xT)}$ và một giả thuyết chuỗi từ W. Quyết định MAP chọn giả thuyết có xác suất hậu nghiệm lớn nhất:

$$\mathbf{\hat{W} = arg\ max_W\ P(W \mid X)}$$
$$\hat{W} = \arg\max_W P(X \mid W)\, P(W)$$

P(X) bị bỏ vì không phụ thuộc W. P(X|W) là acoustic-model likelihood; P(W) là language-model prior.

Trong case study từ đơn, mỗi từ w có một HMM λw. Khi không dùng prior, quy tắc trở thành maximum likelihood:

$$\mathbf{\hat{w} = arg\ max_w\ P(X \mid \lambda w)}$$
$$\text{tương đương}\quad \hat{w} = \arg\max_w \log P(X \mid \lambda w)$$

Tính trong miền log giúp tránh underflow. Khi so sánh các model trên cùng một utterance, T giống nhau; khi cần ngưỡng từ chối giữa các utterance khác độ dài có thể dùng log-likelihood/frame.

3.2. Đặc trưng MFCC động: 13 + 13 + 13

Lab 3 kế thừa MFCC từ Lab 2. Một vector tĩnh $c_t \in R^{13}$ mô tả phổ ngắn hạn. Do HMM chuẩn giả định quan sát tại từng thời điểm phụ thuộc vào trạng thái hiện tại và thường bỏ qua phụ thuộc trực tiếp giữa các frame, ta bổ sung đặc trưng động để đưa thông tin biến thiên theo thời gian vào mỗi frame.

$$\mathbf{\Delta c_t \approx c_t - c_{t-1}}$$
$$\Delta^2 c_t \approx \Delta c_t - \Delta c_{t-1}$$
$$x_t = [\ c_t^T\ ,\ \Delta c_t^T\ ,\ \Delta^2 c_t^T\ ]^T \in R^{39}$$

Đây là dạng sai phân đơn giản, phù hợp mục đích giải thích. Thư viện như librosa có thể dùng phép hồi quy nhiều frame để ước lượng đạo hàm ổn định hơn.

Tại sao 39 chiều? 13 MFCC mô tả hình dạng phổ; Δ mô tả xu hướng thay đổi; Δ² mô tả thay đổi của xu hướng. Huang–Acero–Hon và Rabiner–Schafer đều nhấn mạnh MFCC cùng đặc trưng động là front-end phổ biến cho nhận dạng tiếng nói.

3.3. Định nghĩa HMM λ = (A, B, π)

$$\mathbf{A = \{a_{ij}\},\quad a_{ij} = P(q_t = j \mid q_{t-1} = i)}$$
$$\pi_i = P(q_1 = i)$$
$$B = \{b_i(x)\},\quad b_i(x_t) = p(x_t \mid q_t = i)$$

A mô tả động học trạng thái; π mô tả trạng thái bắt đầu; B mô tả xác suất sinh vector âm học ở mỗi trạng thái.

- Giả thiết Markov bậc 1: q_t chỉ phụ thuộc q_{t-1}.
- Giả thiết output independence: với chuỗi trạng thái đã biết, mỗi x_t được phát từ trạng thái q_t.
- Left-to-right HMM: chỉ cho phép self-loop và chuyển sang trạng thái kế tiếp; phản ánh tiến trình phát âm không quay ngược theo thời gian.
- Với whole-word HMM, mỗi model đại diện trực tiếp cho một từ; với subword HMM, state/model đại diện phone hay đơn vị nhỏ hơn.

3.4. Xác suất phát xạ Gaussian

Trong Lab, mỗi trạng thái dùng một Gaussian D chiều. Với D = 39:

$$\mathbf{b_i(x) = \mathcal{N}(x;\ \mu_i,\ \Sigma_i)}$$
$$\mathcal{N}(x;\ \mu,\ \Sigma) = 1\ /\ ((2\pi)^{(D/2)}\ |\Sigma|^{(1/2)})\ \cdot\ \exp[\ -\tfrac{1}{2}\ (x-\mu)^T\ \Sigma^{-1}\ (x-\mu)\ ]$$

μi là vector trung bình; Σi là ma trận hiệp phương sai. Với covariance_type="diag", chỉ giữ phương sai từng chiều, giảm số tham số và cần ít dữ liệu hơn.

$$\mathbf{GMM\ mở\ rộng:\ b_i(x) = \Sigma_k\ c_{ik}\ \mathcal{N}(x;\ \mu_{ik},\ \Sigma_{ik})}$$
$$\Sigma_k\ c_{ik} = 1,\quad c_{ik} \geq 0$$

GMM có khả năng mô tả phân bố phức tạp hơn nhưng cần nhiều dữ liệu và tính toán hơn. Lab cơ bản dùng 1 Gaussian/state.

3.5. Xác suất của một đường trạng thái và chuỗi quan sát

$$\mathbf{P(X, Q \mid \lambda) = \pi_\{q_1\}\ b_\{q_1\}(x_1)\ \cdot\ \prod_\{t=2\}\^\{T\}\ a_\{q_{t-1},q_t\}\ b_\{q_t\}(x_t)}$$

Q=(q1,…,qT) là một đường trạng thái cụ thể. Likelihood P(X\|λ) phải cộng xác suất của tất cả các đường hợp lệ, trong khi Viterbi chỉ giữ đường có xác suất lớn nhất.

3.6. Forward algorithm – tính likelihood P(X|λ)

Định nghĩa $\alpha_t(j)$ là xác suất đã sinh $x_1 \ldots x_t$ và đang ở trạng thái j tại thời điểm t.

$$\mathbf{Khởi\ tạo:\quad \alpha_1(j) = \pi_j\ b_j(x_1)}$$
$$\text{Đệ quy:}\quad \alpha_t(j) = b_j(x_t)\ \Sigma_i\ \alpha_{t-1}(i)\ a_{ij}$$

Kết thúc: $P(X|\lambda) = \Sigma_j \alpha T(j)$

Dynamic programming tái sử dụng các xác suất cục bộ, tránh liệt kê mọi chuỗi trạng thái. Trong thực tế cần scaling hoặc log-domain để tránh số rất nhỏ.

3.7. Backward và Baum-Welch (Forward–Backward)

Định nghĩa $\beta_t(i)$ là xác suất sinh phần quan sát còn lại $x_{t+1}…xT$ khi đang ở state i tại t:

$$\boldsymbol{\beta T(i) = 1}$$
$$\beta_t(i) = \Sigma_j a_{ij}\, b_j(x_{t+1})\, \beta_{t+1}(j)$$

Từ α và β, tính occupancy và transition responsibility:

$$\boldsymbol{\gamma_t(i) = \alpha_t(i)\, \beta_t(i) / P(X|\lambda)}$$
$$\xi_t(i,j) = \alpha_t(i)\, a_{ij}\, b_j(x_{t+1})\, \beta_{t+1}(j) / P(X|\lambda)$$

$\gamma t(i) \approx$ mức "chịu trách nhiệm" của state i cho frame t; $\xi t(i,j) \approx$ mức "chịu trách nhiệm" của chuyển i⟶j tại thời điểm t.

Với một Gaussian/state, M-step có dạng trực quan:

$$\boldsymbol{\pi_i(new) = \gamma_1(i)}$$
$$a_{ij}(new) = \Sigma_t\, \xi_t(i,j) / \Sigma_t\, \gamma_t(i)$$
$$\mu_i(new) = \Sigma_t\, \gamma_t(i)\, x_t / \Sigma_t\, \gamma_t(i)$$
$$\Sigma_i(new) = \Sigma_t\, \gamma_t(i)(x_t-\mu_i)(x_t-\mu_i)^T / \Sigma_t\, \gamma_t(i)$$

Baum-Welch là EM: lặp E-step (tính kỳ vọng bằng Forward–Backward) và M-step (cập nhật tham số) cho đến hội tụ đến một cực đại cục bộ.

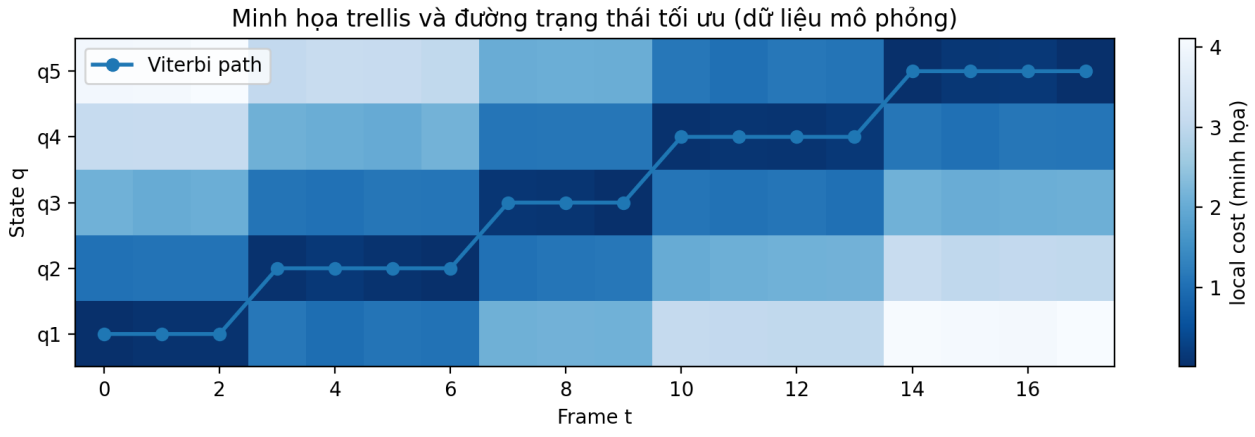
Baum-Welch không đảm bảo tìm cực đại toàn cục. Khởi tạo model, lượng dữ liệu, số state, kiểu covariance và chuẩn hóa đặc trưng ảnh hưởng mạnh đến kết quả.

3.8. Viterbi – tìm chuỗi trạng thái tốt nhất

Forward cộng xác suất của mọi đường; Viterbi thay phép cộng bằng phép max. Định nghĩa $\delta_t(j)$ là xác suất tốt nhất của một đường kết thúc ở state j tại frame t.

$$\boldsymbol{\delta_1(j) = \pi_j\, b_j(x_1)}$$
$$\delta_t(j) = b_j(x_t) \cdot \max_i [\, \delta_{t-1}(i)\, a_{ij} \,]$$
$$\psi_t(j) = \arg\max_i [\, \delta_{t-1}(i)\, a_{ij} \,]$$

ψ lưu back-pointer. Sau frame cuối, chọn state cuối tốt nhất rồi backtrack để tìm path Q. Trong code thường dùng log δ để biến tích thành tổng.*



Hình 2. Trellis mô phỏng: Viterbi tìm một đường đơn đi qua các trạng thái theo thời gian.

3.9. Whole-word HMM và acoustic score

$$\mathbf{s_w(X) = \log P(X \mid \lambda_w)}$$
$$\hat{w}_ML = \arg\max_w s_w(X)$$
$$\bar{s}_w(X) = s_w(X) / T \quad \text{(tùy chọn cho threshold / so sánh khác độ dài)}$$

Trong một trial, tất cả model chấm cùng chuỗi X nên argmax không đổi nếu chia cho T. Score/frame hữu ích khi đặt ngưỡng reject hay xem thống kê giữa các utterance có độ dài khác nhau.

3.10. Lexicon và mô hình ngôn ngữ

Whole-word HMM của case study không cần pronunciation lexicon. Khi chuyển sang subword/phone HMM, lexicon ánh xạ từ sang chuỗi đơn vị phát âm:

```
WORD    PHONE_1 PHONE_2 ... PHONE_K
# Phone symbols phải theo inventory/từ điển phát âm do giảng viên cung cấp.
```

Bài Lab không áp đặt một phiên âm tiếng Việt cụ thể vì còn phụ thuộc hệ phone và phương ngữ. Giảng viên nên cung cấp phone inventory/lexicon chuẩn cho dữ liệu của lớp nếu triển khai phần subword.

Với bigram:

$$\mathbf{P(W) \approx \prod_t P(w_t \mid w_{t-1})}$$
$$\text{MLE: } P(w_j \mid w_i) = C(w_i,w_j) / C(w_i)$$
$$\text{Add-k: } P(w_j \mid w_i) = (C(w_i,w_j)+k) / (C(w_i)+kV)$$

V là kích thước từ vựng. Smoothing tránh xác suất 0 cho cặp từ chưa thấy.

$$\mathbf{Score(W) = \log P_A(X|W) + \lambda_LM \log P_L(W)}$$

λLM dùng để cân bằng thang điểm acoustic và language. Trong case study, đây là re-ranking mang tính sư phạm, chưa phải decoder mạng từ đầy đủ.

3.11. Đánh giá

$$\mathbf{Accuracy = N_correct / N_total \times 100\%}$$
$$\text{WER} = (S + D + I) / N \times 100\%$$

S: substitution; D: deletion; I: insertion; N: số từ tham chiếu. Với isolated-word classification có đúng một từ/trial, accuracy và confusion matrix là trực quan nhất.

- Confusion matrix cho biết cặp lớp nào hay nhầm.
- So sánh train accuracy và test accuracy để phát hiện overfitting.
- Thử dữ liệu người nói khác hoặc thêm nhiễu để quan sát mismatch giữa training và testing.

3.12. DTW và HMM – mối quan hệ và khác biệt

Khía cạnh	DTW	HMM
Biểu diễn	Template chuỗi vector	Mô hình xác suất trạng thái
Biến thiên tốc độ	Warping path trực tiếp	Self-loop/transition duration ngầm
Huấn luyện	Lưu/chọn template	Ước lượng A, B, π từ nhiều utterance
Score	Distance/distortion	Likelihood/log-likelihood
Dữ liệu	Hoạt động tốt với ít mẫu	Cần đủ dữ liệu cho tham số ổn định
Mở rộng	Từ vựng nhỏ	Kết hợp subword, lexicon, LM, decoder
Dynamic programming	Tìm đường chi phí nhỏ nhất	Forward/Viterbi/Baum-Welch

4. Yêu cầu thực hành bắt buộc

4.1. A — Chuẩn hóa dữ liệu và trích đặc trưng

- Đọc WAV, mono, Fs=16 kHz; cắt silence bằng hàm từ Lab 2 hoặc VAD đơn giản.
- Trích 13 MFCC; tính Δ và Δ^2; ghép thành ma trận $T \times 39$.
- Thực hiện CMVN theo utterance hoặc theo tập train; không dùng thống kê của test để chuẩn hóa train.
- Vẽ heatmap MFCC/Δ/Δ^2 của ít nhất 2 từ và mô tả khác biệt.

4.2. B — Khởi tạo HMM trái-sang-phải

- Chọn 4 hoặc 5 state/word; chỉ cho self-loop và next-state transition.
- Đặt $\pi=[1,0,\ldots,0]$. Khởi tạo $a_{ii} \approx 0.6–0.8$, $a_{i,i+1} \approx 1 - a_{ii}$; state cuối self-loop=1.
- Dùng Gaussian diagonal covariance. Giải thích tại sao diagonal covariance phù hợp hơn full covariance khi dữ liệu ít.
- Sau fit, in transmat_ và kiểm tra topology cấm chuyển ngược/skip không bị phá.

4.3. C — Huấn luyện một HMM cho mỗi lớp

- Nối các utterance train của cùng một lớp thành X_concat và truyền lengths cho hàm fit.
- Theo dõi convergence monitor/log-likelihood nếu thư viện hỗ trợ.
- Lưu model cùng label; không dùng test file trong quá trình fit hoặc lựa chọn tham số cuối cùng.
- Thử n_states $\in \{3,5,7\}$; báo cáo hiện tượng underfit/overfit hoặc lỗi hội tụ.

4.4. D — Nhận dạng và Viterbi alignment

- Tính score($X|\lambda_w$) cho mọi lớp; chọn argmax.
- Dùng decode(..., algorithm="viterbi") để lấy state path của ít nhất 2 utterance.
- Vẽ state index theo frame; liên hệ số frame ở mỗi state với duration của các đoạn âm học.
- Tạo bảng score của một test utterance đối với tất cả model để giải thích quyết định.

4.5. E — So sánh với DTW baseline

- Chạy bộ nhận dạng MFCC+DTW của Lab 2 trên cùng train/test split.
- So sánh accuracy, confusion matrix, tốc độ và số mẫu huấn luyện cần thiết.
- Chỉ ra ít nhất một trường hợp HMM tốt hơn DTW hoặc ngược lại và giải thích bằng đặc tính dữ liệu.

4.6. F — Lexicon mini

- Tạo file lexicon.txt theo format WORD PHONE_1 ... PHONE_K với phone sequence do giảng viên cung cấp.
- Viết code parser lexicon thành dict.
- Giải thích cách word HMM có thể được tạo bằng cách nối phone HMM theo lexicon; không bắt buộc huấn luyện phone HMM trong phần cơ bản.

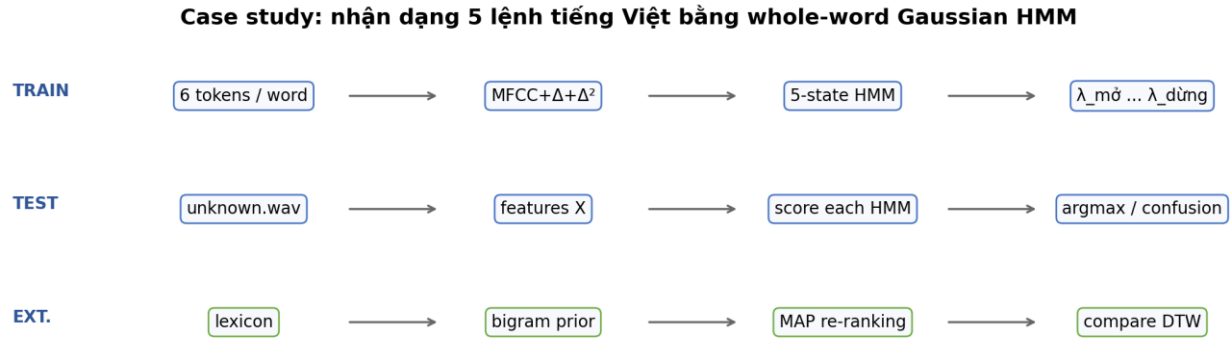
4.7. G — Bigram và MAP re-ranking

- Từ một danh sách câu/lệnh nhỏ do giảng viên cung cấp, đếm unigram và bigram.
- Tính bigram có add-k smoothing.
- Với một utterance có hai acoustic candidate gần nhau, dùng previous-word context và λLM để re-rank.
- Thử ít nhất 3 giá trị λLM và giải thích khi LM quá mạnh có thể lấn át bằng chứng âm học.

4.8. H — Đánh giá và thí nghiệm robustness

- Báo cáo accuracy và confusion matrix trên test set sạch.
- Thử ít nhất một điều kiện mismatch: người nói mới, khoảng cách microphone khác, hoặc noise nhẹ.
- So sánh score distribution của đúng lớp và sai lớp; đề xuất reject threshold nếu hệ thống cần nhận “unknown”.

5. Case study — 5 lệnh tiếng Việt bằng Gaussian HMM



Hình 3. Quy trình case study: huấn luyện whole-word HMM, test, rồi mở rộng lexicon/LM.

5.1. Mục tiêu case study

Xây một recognizer cho 5 lệnh: **mở, đóng, lên, xuống, dừng**. Mỗi từ có 6 file train và 2 file test. Hệ thống sử dụng 39-D MFCC động và 5-state left-to-right Gaussian HMM. Đây là cấu hình sư phạm; không phải cấu hình tối ưu cho tiếng Việt hay cho sản phẩm thực tế.

5.2. Bước 1 — Trích đặc trưng 39 chiều

```
import numpy as np
import librosa

def extract_feat(path, sr=16000):
    y, sr = librosa.load(path, sr=sr, mono=True)
    y, _ = librosa.effects.trim(y, top_db=30)
    y = librosa.effects.preemphasis(y, coef=0.97)

    mfcc = librosa.feature.mfcc(
        y=y, sr=sr, n_mfcc=13,
        n_fft=512, win_length=int(0.025*sr),
        hop_length=int(0.010*sr), window='hamming',
        n_mels=26, fmin=20, fmax=sr/2
    )
    d1 = librosa.feature.delta(mfcc, order=1)
    d2 = librosa.feature.delta(mfcc, order=2)
    F = np.vstack([mfcc, d1, d2]).T   # T x 39

    # utterance-level CMVN
    mu = F.mean(axis=0, keepdims=True)
    sd = F.std(axis=0, keepdims=True) + 1e-8
    return (F - mu) / sd
```

Kiểm tra In shape của 3 file. Chiều thứ hai phải bằng 39. Số frame T có thể khác giữa các utterance; HMM xử lý trực tiếp chuỗi có độ dài biến đổi.

5.3. Bước 2 — Tạo HMM trái-sang-phải

```
import numpy as np
from hmmlearn.hmm import GaussianHMM

def make_lr_hmm(n_states=5, n_iter=80):
    model = GaussianHMM(
        n_components=n_states,
        covariance_type='diag',
        n_iter=n_iter, tol=1e-3,
        min_covar=1e-3,
        init_params='mc',   # means/covars: thư viện khởi tạo
        params='tmc',       # cập nhật transition, means, covars
        random_state=0
    )

    start = np.zeros(n_states)
    start[0] = 1.0
    A = np.zeros((n_states, n_states))
    for i in range(n_states - 1):
        A[i, i] = 0.7
        A[i, i+1] = 0.3
```

```
    A[-1, -1] = 1.0

    model.startprob_ = start
    model.transmat_ = A
    return model
```

Lưu ý phiên bản thư viện hmmlearn có thể thay đổi hành vi giữa các phiên bản. Sau model.fit(...), luôn in startprob_ và transmat_. Nếu các transition bị cấm (giá trị 0) trở thành khác 0, hãy cố định transition bằng params="mc" hoặc dùng cơ chế constraint phù hợp với phiên bản của lớp.

5.4. Bước 3 — Huấn luyện một model cho mỗi từ

```
from pathlib import Path
import numpy as np

labels = ['mo', 'dong', 'len', 'xuong', 'dung']
models = {}

for label in labels:
    feats = []
    lengths = []
    for wav in sorted(Path('dataset/train', label).glob('*.wav')):
        F = extract_feat(str(wav))
        feats.append(F)
        lengths.append(len(F))

    X = np.vstack(feats)
    model = make_lr_hmm(n_states=5)
    model.fit(X, lengths)
    models[label] = model

    print(label, model.monitor_.converged)
    print(np.round(model.transmat_, 3))
```

- Nếu warning “degenerate solution” xuất hiện, giảm số state, tăng dữ liệu hoặc đơn giản hóa feature/covariance.
- Nếu một lớp có quá ít frame cho 5 state, model có thể không ổn định.
- Không đánh giá bằng các file vừa dùng để fit.

5.5. Bước 4 — Nhận dạng maximum likelihood

```
def recognize(path, models):
    F = extract_feat(path)
    scores = {label: model.score(F) for label, model in models.items()}
    pred = max(scores, key=scores.get)
    return pred, scores

pred, scores = recognize('dataset/test/mo/mo_07.wav', models)
print('prediction =', pred)
for k,v in sorted(scores.items(), key=lambda z: z[1], reverse=True):
    print(f'{k:6s}: {v:10.2f}')
```

Yêu cầu: chọn ít nhất 3 test utterance, lập bảng acoustic score với 5 model. Giải thích vì sao model có score lớn nhất được chọn và kiểm tra khoảng cách score giữa top-1 và top-2.

5.6. Bước 5 — Xem Viterbi state alignment

```
F = extract_feat('dataset/test/mo/mo_07.wav')
logp, states = models['mo'].decode(F, algorithm='viterbi')
print('Viterbi log-prob =', logp)
print('state sequence =', states)

import matplotlib.pyplot as plt
plt.step(range(len(states)), states, where='post')
plt.xlabel('Frame')
plt.ylabel('State')
plt.title('Viterbi alignment: mo_07.wav vs model MO')
plt.show()
```

Phân tích Một left-to-right model hợp lý thường cho state path không giảm. Duration ở state được biểu hiện bằng số frame liên tiếp giữ cùng state. Không nên diễn giải mỗi state là một phoneme cụ thể nếu chưa huấn luyện/giám sát ở mức phoneme.

5.7. Bước 6 — Confusion matrix và accuracy

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, accuracy_score
import matplotlib.pyplot as plt

true_y, pred_y = [], []
for label in labels:
    for wav in sorted(Path('dataset/test', label).glob('*.wav')):
        pred, _ = recognize(str(wav), models)
        true_y.append(label)
        pred_y.append(pred)

print('accuracy =', accuracy_score(true_y, pred_y))
cm = confusion_matrix(true_y, pred_y, labels=labels)
ConfusionMatrixDisplay(cm, display_labels=labels).plot(cmap='Blues')
plt.show()
```

5.8. Bước 7 — So sánh HMM với DTW

Chỉ số	DTW (Lab 2)	HMM (Lab 3)	Nhận xét
Accuracy			
Số mẫu train/lớp			
Thời gian train			
Thời gian test/trial			
Top confusion pair			
Khả năng giải thích alignment	warping path	Viterbi state path	

5.9. Bước 8 — Bigram re-ranking (mở rộng)

Giả sử previous word đã biết và ta cần chọn từ hiện tại. Tạo bigram counts từ corpus nhỏ, rồi cộng prior vào acoustic score/frame:

```
import math

def bigram_prob(prev, cur, counts2, counts1, V, k=0.5):
    return (counts2.get((prev,cur),0) + k) / (counts1.get(prev,0) + k*V)

def rerank(F, prev, models, counts2, counts1, lm_weight=0.3):
    T = len(F)
    total = {}
    for w, m in models.items():
        a = m.score(F) / T
        lm = math.log(bigram_prob(prev, w, counts2, counts1, len(models)))
        total[w] = a + lm_weight * lm
    return max(total, key=total.get), total
```

- Chọn một utterance mà top-1 và top-2 acoustic scores gần nhau.
- Thử λLM = 0, 0.1, 0.3, 1.0.
- Ghi lại khi nào LM giúp sửa lỗi và khi nào LM làm sai một quyết định âm học vốn đúng.
- Nhấn mạnh: full continuous-speech decoder cần tìm đồng thời word boundaries và chuỗi từ; re-ranking này chỉ là minh họa nguyên lý Bayes.

6. Thí nghiệm bắt buộc và câu hỏi phân tích

Thí nghiệm	Biến thay đổi	Đại lượng quan sát	Câu hỏi
E1	n_states = 3,5,7	accuracy, convergence	Bao nhiêu state là hợp lý với lượng dữ liệu hiện có?
E2	MFCC 13 vs MFCC+Δ+Δ²	accuracy/confusion	Đặc trưng động giúp lớp nào rõ nhất?
E3	diag vs full covariance (nếu đủ data)	train/test score	Full covariance có overfit không?
E4	DTW vs HMM	accuracy/time	Mô hình nào tốt hơn với ít mẫu?
E5	speaker/environment mismatch	accuracy drop	Nguồn mismatch nào gây giảm mạnh nhất?
E6	λLM	top-1 decision	Khi nào LM lấn át acoustic evidence?

6.1. Câu hỏi báo cáo

- Forward và Viterbi khác nhau ở phép toán nào và trả lời hai câu hỏi khác nhau ra sao?
- Giải thích $\gamma t(i)$ và $\xi t(i,j)$ bằng ngôn ngữ "trách nhiệm" trong EM.
- Tại sao Baum-Welch có thể hội tụ đến nghiệm xấu? Liệt kê ít nhất ba nguyên nhân thực nghiệm.
- Tại sao diagonal covariance thường được ưu tiên khi dữ liệu nhỏ và feature dimension cao?
- Vì sao delta features hữu ích dù HMM đã mô hình hóa chuỗi theo thời gian?
- Whole-word HMM không cần lexicon trong trường hợp nào? Khi nào lexicon trở thành bắt buộc?
- Nếu test utterance dài hơn do nói chậm, self-loop của HMM giúp gì?
- Với cùng một utterance, có cần chia log-likelihood cho T để chọn model tốt nhất không? Tại sao?
- So sánh warping path của DTW với state path của Viterbi.
- Nếu hệ thống gặp một từ ngoài vocabulary, tại sao argmax vẫn bắt buộc chọn một lớp? Hãy đề xuất reject criterion.

7. Sản phẩm nộp và tiêu chí chấm

7.1. Sản phẩm nộp

- Lab03.ipynb hoặc Lab03.py chạy từ đầu đến cuối trong môi trường được quy định.
- Thư mục models/ hoặc file serialized model (nếu giảng viên yêu cầu).
- Confusion matrix, bảng likelihood, ít nhất 2 Viterbi path plots và bảng DTW-vs-HMM.
- lexicon.txt theo inventory do giảng viên cung cấp; corpus/counts dùng cho bigram extension.
- Báo cáo PDF/DOCX ngắn 4–6 trang: thiết kế, tham số, kết quả, phân tích lỗi, kết luận.
- README ghi cách chạy và cấu trúc dữ liệu; không nộp dataset lớn nếu giảng viên không yêu cầu.

7.2. Rubric 10 điểm

Hạng mục	Điểm	Mức đạt
Chuẩn bị dữ liệu + MFCC/Δ/Δ^2 đúng	1.5	Đúng split, đúng shape, không leakage train/test
Hiểu/cấu hình HMM	2.0	Topology, Gaussian, tham số hợp lý; giải thích đúng
Huấn luyện + likelihood + Viterbi	2.0	Pipeline chạy; có alignment và score analysis
Đánh giá + so sánh DTW	1.5	Accuracy, confusion, baseline công bằng
Lexicon + LM extension	1.0	Đúng format; bigram/re-ranking có thí nghiệm
Thí nghiệm & phân tích lỗi	1.0	Có kiểm soát biến, nhận xét dựa trên số liệu
Mã nguồn, báo cáo, tái lập	1.0	Rõ ràng, chạy lại được, hình/bảng có chú thích

8. Lỗi thường gặp và gợi ý xử lý

Hiện tượng	Nguyên nhân thường gặp	Cách xử lý
HMM không hội tụ	Quá nhiều state/parameter, quá ít data	Giảm n_states; tăng token; dùng diag covariance
Score = −inf / NaN	Covariance quá nhỏ, dữ liệu lỗi, numerical underflow	CMVN; min_covar; kiểm tra NaN/inf; dùng log-domain
Model dự đoán một lớp cho mọi input	Khởi tạo xấu hoặc class imbalance	Kiểm tra train files, features, transitions, means
Train rất cao, test thấp	Overfitting / leakage / speaker mismatch	Tách test đúng; giảm complexity; thêm dữ liệu đa dạng
Viterbi path nhảy ngược	Topology không được giữ sau fit	Kiểm tra transmat_; cố định/constraint transition
LM làm accuracy giảm	LM weight quá lớn hoặc corpus không phù hợp	Giảm λLM; kiểm tra smoothing/corpus
Không chạy hmmlearn	Khác phiên bản API	Dùng environment do giảng viên cung cấp; đọc version và điều chỉnh params/init_params

9. Bài mở rộng

- Thay 1 Gaussian/state bằng GMM 2–4 components và đánh giá khi nào có lợi.
- Huấn luyện phone/subword HMM với lexicon do giảng viên cung cấp; tạo word HMM bằng concatenation.
- Thêm silence model và thực hiện nhận dạng chuỗi hai từ bằng một mạng HMM nhỏ + Viterbi.
- Thêm unknown/reject bằng threshold trên normalized log-likelihood hoặc khoảng cách top-1/top-2.
- So sánh speaker-dependent và leave-one-speaker-out evaluation.
- Thử feature normalization ở mức utterance và global-train CMVN; phân tích channel mismatch.

10. Tài liệu tham khảo và phạm vi nguồn

- X. Huang, A. Acero, H.-W. Hon, Spoken Language Processing, Prentice-Hall, 2001. Các phần liên quan: MFCC/dynamic features, HMM, acoustic modeling, language modeling, search.
- L. R. Rabiner, R. W. Schafer, Theory and Applications of Digital Speech Processing, Pearson/Prentice-Hall, 2011 (đề cương CSE457 liệt kê bản 2010). Các phần liên quan: MFCC/cepstrum, HMM/ASR overview, isolated digit recognition.
- D. Jurafsky, J. H. Martin, Speech and Language Processing, 2nd ed., Prentice-Hall, 2008. Tài liệu tham khảo của học phần cho speech recognition và language modeling.
- Đề cương chi tiết học phần CSE457 — Xử lý âm thanh và tiếng nói, Trường Đại học Thủy lợi, 2023.